Assignment-2

Linear Data Structures

IMPLEMENTATION OF DIFFERENT DATA STRUCTURES

Institute Name- K.R. Mangalam University

Course Name: Data Structures

Program Name: B. Tech Computer Science and Engineering (AI and ML)

Faculty Name: Dr. Neetu Chauhan

BUILDING A PYTHON AERT TO LEARN DIFFERENT DATA STRUCTURES

By: Mehul Srivastava

Roll No.- 2501730149

1. Introduction:

This assignment focuses on implementing a toolkit called Linear Data Structures Toolkit (LDST) using Python. The aim is to understand how basic linear data structures like linked list work internally and how operations are performed on them. The toolkit includes:

1. Dynamic Array implementation
2. Singly Linked List
3. Doubly Linked List
4. Stack and Queue using Linked List
5. Balanced Parentheses Checker

2. Dynamic Array:

2.1. What is Dynamic Array

A dynamic array is a resizable array that increases its capacity when it becomes full. Unlike static arrays, it can grow during runtime.

2.2. Why resizing is required?

When the array becomes full, a new array of double capacity is created and all elements are copied into it. This allows the array to store more elements without overflow.

2.3. Why append has amortized time complexity of O(1)?

In a dynamic array, most append() operations take O(1) time because the element is simply added at the end. However, when the array becomes full, its size is increased and all elements are copied to a new larger array. This copying takes O(n) time.

Since resizing does not happen frequently, the overall average time per append operation remains O(1). This average performance over multiple operations is called amortized time complexity.

3. Linked List:

3.1. Singly Linked List:

3.1.1. What is Singly Linked List?

A singly linked list consists of nodes where each node contains data and a pointer to the next node.

3.2. Doubly Linked List:

3.2.1. What is Doubly Linked List?

A doubly linked list has nodes with two pointers. One pointing to the next node and one pointing to the previous node

3.3. Stack Operations and their time complexities:

Stack follows the Last In First Out (LIFO) principle.

push(): O(1)
Adding an element is done at the top, so no traversal is needed.

pop(): O(1)
Removing the top element is direct and does not require searching.

peek(): O(1)
The top element is accessed directly without modifying the stack.

3.4. Queue Operations and their time complexities:

Queue follows the First In First Out (FIFO) principle.

enqueue(): O(1)
Element is added at the rear using a tail pointer.

dequeue(): O(1)
Element is removed from the front directly.

front(): O(1)
The front element is accessed without traversal.

4. Balanced Parathesis Checker:

4.1. Why?

To check whether an expression contains balanced brackets.

4.2. Implementation using stack:

- Push opening brackets onto stack
- For closing brackets, check top element
- If mismatch occurs → not balanced